COMPLETE ARCHITECTURE & CODE MANUAL

GA-HRMS Backend Platform

Production-Grade Enterprise Multi-Tenant HRMS | Architecture, Flow, & Code Reference

Stack: Java 21 LTS | Spring Boot 3.4.1 | PostgreSQL 16

Author: Dipratna Mahire

Version: Phase 0, 1 & 2 Complete

1. Architecture & Design Decisions

GA-HRMS ek production-grade Human Resource Management System hai jo **Modular Monolith** pattern par bana hai:

- **Modular Monolith Kyu?** Microservices me deployment, network latency, aur distributed transactions ki bohot zyada complexity hoti hai. Isliye monolithic repository me har module (Identity, Organization, Employee, Attendance, Payroll) ko alag package me strictly isolate kiya gaya hai.
- **Multi-Tenant Baseline:** Database me ek hi table multiple companies ko serve karti hai. Har table me `company_id` se data alag rehta hai.
- **Effective Dating & Immutability:** Company ya policy ke badalne se purana historical attendance aur salary data kabhi corrupt nahi hota.

> **Key Golden Rule:** Controller me sirf HTTP request receive hoti hai aur validation hota hai. Sara business logic `Service` class me hota hai, aur database queries `Repository` handle karta hai.

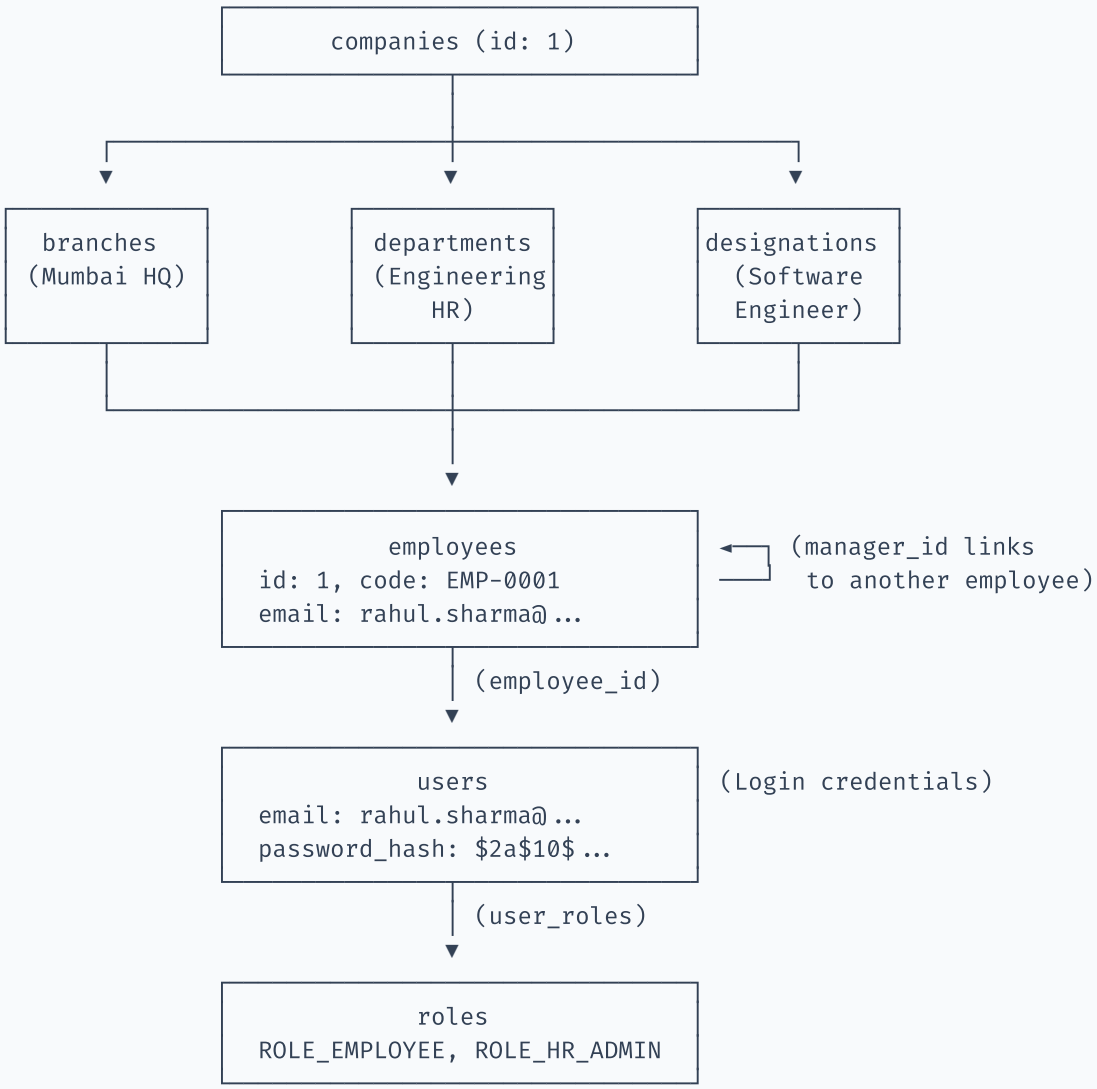
2. Folder Structure & Responsibilities

Har folder ka ek specific kaam hai:

```
ga-hrms-backend/
├── 📁 config/
│   ├── ApplicationSecurityConfig.java  ⟶ PasswordEncoder, UserDetailsService, Authent
│   ├── SecurityConfig.java             ⟶ SecurityFilterChain (CORS, CSRF, URL Protect
│   ├── JwtAuthenticationFilter.java    ⟶ Har incoming request se Bearer token verify
│   └── OpenApiConfig.java              ⟶ Swagger UI / OpenAPI 3 settings
│
├── 📁 common/                           ⟶ Shared DTOs & Error Handling
│   ├── dto/ApiResponse.java            ⟶ Standard Response: {success, message, data,
│   ├── dto/ErrorResponse.java          ⟶ Standard Error: {status, errorCode, message,
│   ├── dto/PageResponse.java           ⟶ Pagination Helper (totalElements, totalPages
│   ├── entity/BaseEntity.java          ⟶ id, createdAt, updatedAt, version (Saari tab
│   └── exception/                      ⟶ GlobalExceptionHandler, ConflictException, R
│
├── 📁 modules/
│   ├── 📁 identity/                     ⟶ Authentication & Roles
│   │   ├── AuthController.java         ⟶ /api/v1/auth/login, /refresh, /logout, /me
│   │   ├── AuthService.java            ⟶ Login verification, JWT issue karna
│   │   └── entity/                     ⟶ User, Role, Permission, RefreshToken
│   │
│   ├── 📁 organization/                 ⟶ Company Hierarchy
│   │   ├── OrganizationController.java ⟶ /companies, /branches, /departments, /design
│   │   ├── OrganizationService.java    ⟶ Hierarchy tree aur unique checks
│   │   └── entity/                     ⟶ Company, Branch, Department, Designation
│   │
│   └── 📁 employee/                     ⟶ Employee Lifecycle (Phase 2)
│       ├── EmployeeController.java     ⟶ /api/v1/employees (Onboard, Directory, Profi
│       ├── EmployeeService.java        ⟶ Auto Employee Code, Auto User Account, Loop
│       ├── EmployeeRepository.java     ⟶ Spring Data JPA queries
│       └── EmployeeSpecification.java  ⟶ Dynamic JPA Search Filters
│
└── 📁 src/main/resources/
    ├── application.yml                 ⟶ Port 8082, JWT secret key (24hr expiry)
    ├── application-dev.yml             ⟶ PostgreSQL 5434 DB Connection
    └── db/migration/                   ⟶ Flyway V1 (18 Tables) & V2 (Seed Admin Data)
```

3. Database Tables Relationship (ER Diagram)

Yeh dekhiye ki tables aapas me kaise connected hain:


companies (id: 1)
branches (Mumbai HQ)
departments (Engineering HR)
designations (Software Engineer)
employees
id: 1, code: EMP-0001
email: rahul.sharma@...
(manager_id links to another employee)
(employee_id)
users
email: rahul.sharma@...
password_hash: $2a$10$...
(Login credentials)
(user_roles)
roles
ROLE_EMPLOYEE, ROLE_HR_ADMIN

4. Deep Dive: Object Creation & Setters Approach

Aapne pucha tha ki **"Service me Object kaha ban raha hai aur value kaha set ho rahi hai?"**

Code me standard, traditional **new keyword + Setter methods** use hue hain taaki code crystal-clear rahe:

```
// 1. Database se Foreign Key Entities fetch ki
Company company = companyRepository.findById(request.getCompanyId()).orElseThrow( ... );
Department department = departmentRepository.findById(request.getDepartmentId()).orElseThrow( ... );

// 2. new keyword se Object BAN RAHA HAI aur SETTERS se VALUES SET HO RAHI HAIN:
Employee employee = new Employee();               // ⟵ YAHA JAVA MEMORY ME OBJECT CREATE HUA!
employee.setEmployeeCode(employeeCode);           // ⟵ VALUE SET HO RAHI HAI
employee.setFirstName(request.getFirstName());    // ⟵ VALUE SET HO RAHI HAI
employee.setLastName(request.getLastName());      // ⟵ VALUE SET HO RAHI HAI
employee.setEmail(request.getEmail());            // ⟵ VALUE SET HO RAHI HAI
employee.setJoiningDate(request.getJoiningDate());// ⟵ VALUE SET HO RAHI HAI
employee.setCompany(company);                     // ⟵ Company Entity SET hui
employee.setDepartment(department);               // ⟵ Department Entity SET hui
employee.setDesignation(designation);             // ⟵ Designation Entity SET hui
employee.setManager(manager);                     // ⟵ Reporting Manager SET hua

// 3. Database me Permanent Save:
Employee savedEmployee = employeeRepository.save(employee); // ⟵ SQL INSERT execute hua

// 4. Response DTO Object Creation & Setting (mapToResponse):
EmployeeResponse response = new EmployeeResponse(); // ⟵ DTO OBJECT CREATE HUA!
response.setId(savedEmployee.getId());              // ⟵ ID SET
response.setEmployeeCode(savedEmployee.getEmployeeCode()); // ⟵ Code SET
response.setFullName(savedEmployee.getFullName());  // ⟵ Name SET
response.setDepartmentName(savedEmployee.getDepartment().getName()); // ⟵ Dept Name SET
return response;                                    // ⟵ Controller ko return
```

> **Fayda:** Is tarike se bilkul clearly dikhta hai ki kab `new Employee()` ya `new EmployeeResponse()` se blank object bana, aur kab ek-ek line me `.setXYZ( ... )` se values assign hui!

5. Request Lifecycle: Postman se Database Tak

Jab aap Postman se `POST /api/v1/employees` request bhejte hain, to step-by-step kya hota hai:

1. **Client:** Postman Header me JWT token bhejta hai: `Authorization: Bearer <token>`

2. **JwtAuthenticationFilter:** Filter token ko intercept karta hai, JJWT library se secret key verify karta hai, aur authenticated user ko Spring Security Context me store karta hai.
3. **Authorization Check:** `@PreAuthorize("hasAnyRole('SUPER_ADMIN', 'HR_ADMIN')")` check karta hai ki kya user ke paas Admin authority hai. Agar nahi, to `403 Forbidden` return ho jata hai.
4. **Controller Validation:** `@Valid @RequestBody CreateEmployeeRequest request` se check hota hai ki first name, email, joining date empty to nahi hai.
5. **Service Business Logic:** `EmployeeService` email duplication check karta hai, unique Employee Code generate karta hai, aur entity save karta hai.
6. **Database Save:** Spring Data JPA PostgreSQL me `INSERT INTO employees` aur `INSERT INTO users` run karta hai.
7. **JSON Output:** Service response DTO banata hai, aur Jackson usko JSON me convert karke HTTP 201 Created return karta hai.

6. Complete Working APIs Catalogue

Saare APIs active port **8082** par live hain:

Module	Method	Endpoint	Access Role	Description
Auth	POST	`/api/v1/auth/login`	Public	Email + Password bhejo, JWT Tokens pao
Auth	POST	`/api/v1/auth/refresh`	Public	Refresh token se naya access token generate karo
Auth	GET	`/api/v1/auth/me`	Authenticated	Logged-in user ka profile summary dekho
Organization	GET	`/api/v1/organization/companies`	Authenticated	Registered companies ki list
Organization	POST	`/api/v1/organization/branches`	Admin / HR	Naya office/branch location create karo
Organization	POST	`/api/v1/organization/departments`	Admin / HR	IT, HR, Finance departments add karo
Organization	POST	`/api/v1/organization/designations`	Admin / HR	Designations (Software Engineer, Manager) add karo
Employee	POST	`/api/v1/employees`	Admin / HR	Employee onboarding + Auto login user account
Employee	GET	`/api/v1/employees`	Admin / HR / Manager	Search, filter aur paginated employee directory

Module	Method	Endpoint	Access Role	Description
Employee	GET	`/api/v1/employees/{id}`	Authenticated	Single employee ka complete profile
Employee	PUT	`/api/v1/employees/{id}`	Admin / HR	Employee details modify karo
Employee	PATCH	`/api/v1/employees/{id}/status`	Admin / HR	Status change (Active/Separated) + User sync

7. Live Verification Test Results

Humne live database me ye test successfully run kiya:

✅ **Test 1: Admin Authentication**
- User: `admin@gahrms.com` | Password: `Admin@123456`
- Status: **HTTP 200 OK** | Token Issued with `ROLE_SUPER_ADMIN`.

✅ **Test 2: Employee Onboarding**
- Created: Rahul Sharma (`rahul.sharma@gahrms.com`)
- Assigned Code: `EMP-0001` | Department: Software Engineering
- Auto User Created: `userId: 2` linked to Employee ID 1.

✅ **Test 3: New Employee Login**
- User: `rahul.sharma@gahrms.com` | Password: `Welcome@123`
- Status: **HTTP 200 OK** | Successfully authenticated with `ROLE_EMPLOYEE`!

8. Roadmap: Agla Module Kya Hai?

Ab hamari organization aur employee master tables ready hain. Ab hum aage ke modules bana sakte hain:

1. **Phase 3 (Next): Time & Attendance ➔** Daily Punch-in/Punch-out, Shift assignment, Grace periods, aur Regularization request workflow.
2. **Phase 4: Leave Management ➔** Leave Types (Casual, Sick, Earned), Balance Ledger, aur Manager Approval Workflow.
3. **Phase 5: Payroll Engine ➔** Salary components, monthly pay calculation, aur locked payslip generation.